

Apache Flink

What is Apache Flink? — Applications

Apache Flink is a framework for stateful computations over unbounded and bounded data streams. Flink provides multiple APIs at different levels of abstraction and offers dedicated libraries for common use cases.

Here, we present Flink's easy-to-use and expressive APIs and libraries.

Building Blocks for Streaming Applications

The types of applications that can be built with and executed by a stream processing framework are defined by how well the framework controls *streams*, *state*, and *time*. In the following, we describe these building blocks for stream processing applications and explain Flink's approaches to handle them.

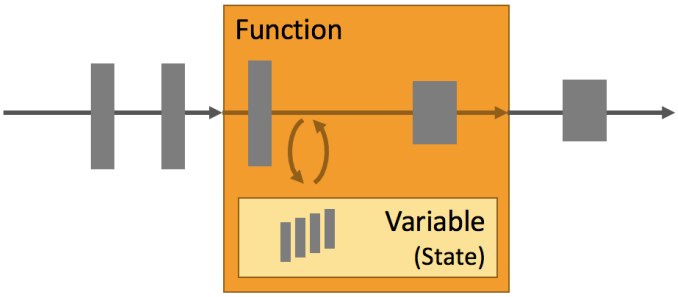
Streams

Obviously, streams are a fundamental aspect of stream processing. However, streams can have different characteristics that affect how a stream can and should be processed. Flink is a versatile processing framework that can handle any kind of stream.

- **Bounded** and **unbounded** streams: Streams can be unbounded or bounded, i.e., fixed-sized data sets. Flink has sophisticated features to process unbounded streams, but also dedicated operators to efficiently process bounded streams.
- **Real-time** and **recorded** streams: All data are generated as streams. There are two ways to process the data. Processing it in real-time as it is generated or persisting the stream to a storage system, e.g., a file system or object store, and processed it later. Flink applications can process recorded or real-time streams.

State

Every non-trivial streaming application is stateful, i.e., only applications that apply transformations on individual events do not require state. Any application that runs basic business logic needs to remember events or intermediate results to access them at a later point in time, for example when the next event is received or after a specific time duration.



Application state is a first-class citizen in Flink. You can see that by looking at all the features that Flink provides in the context of state handling.

- **Multiple State Primitives**: Flink provides state primitives for different data structures, such as atomic values, lists, or maps. Developers can choose the state primitive that is most efficient based on the access pattern of the function.
- **Pluggable State Backends**: Application state is managed in and checkpointed by a pluggable state backend. Flink features different state backends that store state in memory or in RocksDB, an efficient embedded on-disk data store. Custom state backends can be plugged in as well.
- **Exactly-once state consistency**: Flink's checkpointing and recovery algorithms guarantee the consistency of application state in case of a failure. Hence, failures are transparently handled and do not affect the correctness of an application.
- **Very Large State**: Flink is able to maintain application state of several terabytes in size due to its asynchronous and incremental checkpoint algorithm.
- **Scalable Applications**: Flink supports scaling of stateful applications by redistributing the state to more or fewer workers.

Time

Time is another important ingredient of streaming applications. Most event streams have inherent time semantics because each event is produced at a specific point in time. Moreover, many common stream computations are based on time, such as windows aggregations, sessionization, pattern detection, and time-based joins. An important aspect of stream processing is how an application measures time, i.e., the difference between event-time and processing-time.

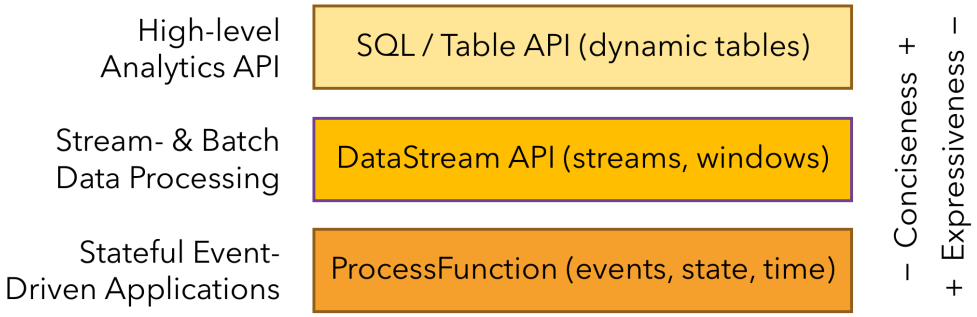
Flink provides a rich set of time-related features.

- **Event-time Mode**: Applications that process streams with event-time semantics compute results based on timestamps of the events. Thereby, event-time processing allows for accurate and consistent results regardless whether recorded or real-time events are processed.
- **Watermark Support**: Flink employs watermarks to reason about time in event-time applications. Watermarks are also a flexible mechanism to trade-off the latency and completeness of results.
- **Late Data Handling**: When processing streams in event-time mode with watermarks, it can happen that a computation was considered completed before all associated events have arrived. Such events are called late events. Flink features multiple options to handle late events, such as rerouting them via side outputs and updating previously completed results.
- **Processing-time Mode**: In addition to its event-time mode, Flink also supports processing-time semantics which performs computations as triggered by the wall-clock time of the processing

machine. The processing-time mode can be suitable for certain applications with strict low-latency requirements that can tolerate approximate results.

Layered APIs

Flink provides three layered APIs. Each API offers a different trade-off between conciseness and expressiveness and targets different use cases.



We briefly present each API, discuss its applications, and show a code example.

The ProcessFunctions

ProcessFunctions are the most expressive function interfaces that Flink offers. Flink provides ProcessFunctions to process individual events from one or two input streams or events that were grouped in a window. ProcessFunctions provide fine-grained control over time and state. A ProcessFunction can arbitrarily modify its state and register timers that will trigger a callback function in the future. Hence, ProcessFunctions can implement complex per-event business logic as required for many stateful event-driven applications.

The following example shows a `KeyedProcessFunction` that operates on a `KeyedStream` and matches `START` and `END` events. When a `START` event is received, the function remembers its timestamp in state and registers a timer in four hours. If an `END` event is received before the timer fires, the function computes the duration between `END` and `START` event, clears the state, and returns the value. Otherwise, the timer just fires and clears the state.

```
/**
* Matches keyed START and END events and computes the difference between
* both elements' timestamps. The first String field is the key attribute,
* the second String attribute marks START and END events.
*/
public static class StartEndDuration
  extends KeyedProcessFunction<String, Tuple2<String, String>, Tuple2<String, Long>>

  private ValueState<Long> startTime;

  @Override
```

```
    public void open(Configuration conf) {
      // obtain state handle
      startTime = getRuntimeContext()
        .getState(new ValueStateDescriptor<Long>("startTime", Long.class));
    }

    /** Called for each processed event. */
    @Override
    public void processElement(
        Tuple2<String, String> in,
        Context ctx,
        Collector<Tuple2<String, Long>> out) throws Exception {

        switch (in.f1) {
          case "START":
            // set the start time if we receive a start event.
            startTime.update(ctx.timestamp());
            // register a timer in four hours from the start event.
            ctx.timerService()
              .registerEventTimeTimer(ctx.timestamp() + 4 * 60 * 60 * 1000);
            break;
          case "END":
            // emit the duration between start and end event
            Long sTime = startTime.value();
            if (sTime != null) {
              out.collect(Tuple2.of(in.f0, ctx.timestamp() - sTime));
              // clear the state
              startTime.clear();
            }
          default:
            // do nothing
        }
    }

    /** Called when a timer fires. */
    @Override
    public void onTimer(
      long timestamp,
      OnTimerContext ctx,
      Collector<Tuple2<String, Long>> out) {

      // Timeout interval exceeded. Cleaning up the state.
      startTime.clear();
    }
}
```

The example illustrates the expressive power of the `KeyedProcessFunction` but also highlights that it is a rather verbose interface.

The DataStream API

The DataStream API provides primitives for many common stream processing operations, such as windowing, record-at-a-time transformations, and enriching events by querying an external data store. The DataStream API is available for Java and is based on functions, such as `map()`, `reduce()`, and `aggregate()`. Functions can be defined by extending interfaces or as Java lambda functions.

The following example shows how to sessionize a clickstream and count the number of clicks per session.

```
// a stream of website clicks
DataStream<Click> clicks = ...

DataStream<Tuple2<String, Long>> result = clicks
  // project clicks to userId and add a 1 for counting
  .map(
    // define function by implementing the MapFunction interface.
    new MapFunction<Click, Tuple2<String, Long>>() {
      @Override
      public Tuple2<String, Long> map(Click click) {
        return Tuple2.of(click.userId, 1L);
      }
    })
  // key by userId (field 0)
  .keyBy(0)
  // define session window with 30 minute gap
  .window(EventTimeSessionWindows.withGap(Time.minutes(30L)))
  // count clicks per session. Define function as lambda function.
  .reduce((a, b) -> Tuple2.of(a.f0, a.f1 + b.f1));
```

SQL & Table API

Flink features two relational APIs, the Table API and SQL . Both APIs are unified APIs for batch and stream processing, i.e., queries are executed with the same semantics on unbounded, real-time streams or bounded, recorded streams and produce the same results. The Table API and SQL leverage Apache Calcite for parsing, validation, and query optimization. They can be seamlessly integrated with the DataStream API and support user-defined scalar, aggregate, and table-valued functions.

Flink's relational APIs are designed to ease the definition of data analytics, data pipelining, and ETL applications.

The following example shows the SQL query to sessionize a clickstream and count the number of clicks per session. This is the same use case as in the example of the DataStream API.

```
SELECT userId, COUNT(*)
FROM clicks
GROUP BY SESSION(clicktime, INTERVAL '30' MINUTE), userId
```

Libraries

Flink features several libraries for common data processing use cases. The libraries are typically embedded in an API and not fully self-contained. Hence, they can benefit from all features of the API and be integrated with other libraries.

- **Complex Event Processing (CEP)** : Pattern detection is a very common use case for event stream processing. Flink's CEP library provides an API to specify patterns of events (think of regular expressions or state machines). The CEP library is integrated with Flink's DataStream API, such that patterns are evaluated on DataStreams. Applications for the CEP library include network intrusion detection, business process monitoring, and fraud detection.

Want to contribute translation?

Edit This Page

flink-packages.org

Apache Software Foundation

License

中文版

Security

Donate

Thanks

Flink blog Github Twitter

The contents of this website are © 2024 Apache Software Foundation under the terms of the Apache License v2. Apache Flink, Flink, and the Flink logo are either registered trademarks or trademarks of The Apache Software Foundation in the United States and other countries.